← Back to the project

PROJECT ARES · SOURCE CODE

CODE HIGHLIGHTS

The most important pieces of code in both NASA HUNCH games, explained in plain English. Each picture is the real code with its file and line numbers. Tap a picture to see it full size.

The big idea: the AI looks after the crew's mental health, but it always asks before it acts, and a human can switch it off at any moment. If something goes wrong while the crew is asleep, the AI stops and a person takes over.

1. Turning sensor readings into a wellbeing score
2. Deciding how worried to be (Tier 0–3), carefully
3. Asking before acting: consent and the “agency test”
4. The human OFF switch
5. Night watch: if anything goes wrong while you sleep, the AI stops
6. Sleeping in your pod: one real minute = one whole night
7. How life on the Moon wears on the crew
8. Keeping the base alive: power, air, water, food and storms
9. ARIA predicts who is about to struggle
10. ARIA tells the commander what to do next, and why
11. Privacy-aware sensors in every zone (centre dorm kept)
12. Mobile controls: the whole game with three buttons

GAME 1 · REALISTIC — MOON BASE FINAL

01 Turning sensor readings into a wellbeing score

Assets/Unity3/Scripts/02_AI_Wellness/Brain.cs — lines 212–236

Turning sensor readings into a wellbeing score

```
212  public void Detect(float dt)
213  {
214      foreach (var c in crew)
215      {
216          if (c.pauseUntilHour > SimClock.hours) { for (int k = 0; k < 6; k++) c.tier[k] = Mathf.Min(c.tier[k], 0); continue; }
217          float confSum = 0, confFull = 0;
218          for (int k = 0; k < 6; k++)
219          {
220              float num = 0, den = 0, full = 0;
221              for (int s = 0; s < 9; s++)
222              {
223                  float w = SensorSpec.W[k, s]; if (w <= 0) continue; full += w;
224                  if (!c.SensorUsable(s)) continue;
225                  if (c.sensors[s] == SensorHealth.Degraded) w *= 0.5f;
226                  w *= c.fbWeight[s];
227                  num += w * Feature(c, (Cond)k, (Sensor)s); den += w;
228              }
229              // voluntary self-check (2 questions, Crew Interface) counts for 24 h as an extra signal for anxiety and depression.
230              // It is the planned fallback when a sensor fails (board Q4): it restores confidence without adding hardware.
231              if ((k == 0 || k == 1) && SimClock.hours - c.selfAt < 24f)
232              {
233                  float v = k == 0 ? c.selfStress : c.selfMood;
234                  if (v >= 0) { float sw = 0.3f * full; num += sw * v; den += sw; }
235              }
236              c.score[k] = den > 0 ? Mathf.Clamp01(num / den) : 0;        // weighted average of every available signal
```

📄 Brain.cs · lines 212–236

What it does. This is where the AI actually reads the crew. Every astronaut has 9 sensors (wristband, sleep tracker, voice, CO_2, movement and so on). For each of the 6 conditions we watch (anxiety, depression, fatigue, conflict, monotony and loss of autonomy), the code takes a weighted average of whichever sensors are working right now.

Why it matters

- A broken sensor is skipped and a glitchy one only counts half, so one bad reading can't panic the system.
- If someone says “that reading is wrong”, `fbWeight` lowers that sensor's say for them. The crew can correct the AI.
- The optional 2-question self-check counts as an extra signal for 24 hours. It's our backup plan when a sensor fails.

When it runs. Every simulated minute, for all 4 crew. It feeds the crew health dashboard, the dome lights and the AI's decisions.

02 Deciding how worried to be (Tier 0–3), carefully

Assets/Unity3/Scripts/02_AI_Wellness/Brain.cs lines 237–262

Deciding how worried to be (Tier 0–3), carefully

```
237     den = Mathf.Min(den, full);                                 // confidence can't exceed 100 %
238     confSum += den; confFull += full;
239     float a = 1 - Mathf.Exp(-dt / CondSpec.TauHours[k]);
240     c.smooth[k] += (c.score[k] - c.smooth[k]) * a;
241     float conf = full > 0 ? den / full : 0;
242     // duration assessment
243     if (c.smooth[k] >= c.Thr(k, CondSpec.T1)) { if (c.aboveSince[k] <= Unset) c.aboveSince[k] = SimClock.hours; }
244     else c.aboveSince[k] = Unset;
245     int target = c.smooth[k] >= c.Thr(k, CondSpec.T3) ? 3 : c.smooth[k] >= c.Thr(k, CondSpec.T2) ? 2 : c.smooth[k] >= c.Thr(k,
    CondSpec.T1) ? 1 : 0;
246     float persistH = CondSpec.PersistHours[k];
247     // tabletop redesign: when the depression signal is CORROBORATED by two independent signals (poor sleep AND low
248     // activity), start silent, reversible Tier-1 support at once and shorten the Tier-2+ persistence 48 h → 36 h.
249     bool corroborated = k == 1 && FastDepression && c.sleepQ < 0.8f && c.mood < 0.6f;
250     // fast lane: the 24 h smoothing lags a fast decline — if the RAW signal has stayed above threshold for 12 h
251     // and activity is clearly low, silent Tier-1 support may start before the smoothed score catches up
252     if (k == 1 && FastDepression) { if (c.score[1] >= CondSpec.T1) { if (!rawSince.ContainsKey(c.id)) rawSince[c.id] =
    SimClock.hours; } else rawSince.Remove(c.id); }
253     bool fastLane = k == 1 && FastDepression && rawSince.TryGetValue(c.id, out float rs0) && SimClock.hours - rs0 >= 12f && c.mood <
    0.5f;
254     if (corroborated || fastLane) persistH = 36f;
255     bool persisted = c.aboveSince[k] > Unset && SimClock.hours - c.aboveSince[k] >= persistH;
256     if (!persisted) target = (fastLane || (corroborated && target >= 1)) ? 1 : 0;
257     if (conf < 0.5f && target > 1) { target = 1; }                          // graceful degradation: never escalate on weak evidence
258     if (EthicsGuard.ReviewOverdue && target > 1) target = 1;
259     int old = c.tier[k];
260     if (target != old) { c.tier[k] = target; OnTierChange(c, k, old, target, conf); }
261 }
262 c.confidence = confFull > 0 ? confSum / confFull : 0;
```

📄 Brain.cs · lines 237–262

What it does. Once there's a score, the AI picks a response tier: 0 = fine, 1 = small silent tweak, 2 = ask the person, 3 = get a human (the medical officer or commander). It only moves up a tier if the score stays high for long enough, not because of one bad hour.

Why it matters

- `persisted` means an issue has to last (for example 48 hours for depression) before the AI gets more involved.
- Graceful degradation: if fewer than half the sensors are trustworthy, it never goes above Tier 1.
- If the weekly human review of the AI is overdue, it's capped at Tier 1 too. Humans stay in charge.

When it runs. Straight after the scores are updated. A tier change calls `OnTierChange`, which starts or stops a response.

03 Asking before acting: consent and the “agency test”

Assets/Unity3/Scripts/02_AI_Wellness/Brain.cs lines 305–334

Asking before acting: consent and the “agency test”

```
305  void RunTier(CrewMember c, int k, int t, float conf, Proto p)
306  {
307      p.start = SimClock.hours; p.monitorUntil = SimClock.hours + CondSpec.MonitorHours[k]; p.startScore = c.smooth[k];
308      // AGENCY TEST (ARES constraint: no intervention that worsens autonomy deprivation)
309      //  • someone already showing autonomy deprivation gets NO silent changes — the AI asks instead
310      //  • nobody gets more than 3 silent changes in 24 h (the habitat must not feel like it is constantly steering you)
311      c.silentAt.RemoveAll(h => SimClock.hours - h > 24f);
312      bool autonomyGuard = k != (int)Cond.Autonomy && c.tier[(int)Cond.Autonomy] >= 1;
313      bool budgetFull = c.silentAt.Count >= 3;
314      if (t == 1 && (autonomyGuard || budgetFull))
315      {
316          Log(c.id, "agency", $"T1 {CondSpec.Name[k]} NOT applied silently — {(autonomyGuard ? "this person already feels over-
     controlled" : "3 silent changes already today")}. Asking instead.", 1, k);
317          if (c.id == playerId && !headless) onToast?.Invoke($"I could adjust your zone to help with {CondSpec.Name[k].ToLower()}.
     Want me to? (Crew Interface → AI actions)");
318          return;
319      }
320      if (t >= 1)
321      {
322          c.silentAt.Add(SimClock.hours);
323          string where = k == 1 && !c.aiPodAccess ? " (shared zones only — pod not invited)" : "";
324          Log(c.id, "tier1", $"T1 {CondSpec.Name[k]}: {T1Text[k, 0]}{where}. Confidence {conf:P0}.", 1, k);
325          onIntervention?.Invoke(Lever[k] + ":1", c.id, k);
326          c.autonomy -= 0.005f;
327          if (c.notifyEverySilentChange && !headless && c.id == playerId) onToast?.Invoke($"FYI: I adjusted your zone
     ({CondSpec.Name[k]} support). You asked to be told every time.");
328      }
329      if (t >= 2)
330      {
331          var o = new Offer { crew = c.id, cond = k, tier = 2, text = T2Offer[k], expires = SimClock.hours + 1f };
332          o.onAccept = () => { Accept(c, k); };
333          o.onDecline = () => { Refuse(c, k, p); };
334          Log(c.id, "tier2", $"T2 {CondSpec.Name[k]}: offer sent — \"{o.text}\"", 2, k);
```

📄 Brain.cs · lines 305–334

What it does. This runs a response. Tier 1 is a small, reversible change, like warmer light or quieter sound in someone's zone. Tier 2 is an offer the person can accept or say no to. Before doing anything quietly, the AI checks it isn't making someone feel controlled.

Why it matters

- If a person already shows signs of feeling over-controlled, the AI doesn't change things silently. It asks instead.
- No one gets more than 3 silent changes in 24 hours, so the habitat never feels like it's steering you.
- AI crewmates decide on offers based on their own autonomy, so saying “no” is part of the simulation.

When it runs. Whenever a crew member moves up a tier. Offers to the player pop up in VR or on screen as Y / N.

04 The human OFF switch

Assets/Unity3/Scripts/02_AI_Wellness/Authority.cs — lines 34–55

The human OFF switch

```
34  public static void Toggle() { if (Blocked) Allow(); else Disallow("the Commander pressed the AI off switch"); }
35
36  public static void Disallow(string why)
37  {
38      if (Blocked) return;
39      Blocked = true; Misbehaving = false;
40      DomeLights.AIControl = false;
41      if (DomeLights.I) DomeLights.I.SetAIControl(false);
42      if (PlayerRig.I) PlayerRig.I.autopilot = false;
43      HabitatEnvironment.I?.Apply("restore:", 1, 0);                        // roll back every AI change, everywhere
44      E?.Log(1, "authority", $"Commander DISALLOWED AI control ({why}). AI is now watch-only: vital-sign alarms stay on; it can't
    change the habitat or take control until a human allows it again.");
45      HUD.I?.Toast("⛔ AI CONTROL DISALLOWED. The AI is now watch-only — it can't change lights, sound, scent or take control. Every
    change it made was undone. (O = allow again)");
46      SpaceRealism.I?.Quindar();
47  }
48
49  public static void Allow()
50  {
51      if (!Blocked) return;
52      Blocked = false;
53      E?.Log(1, "authority", "Commander allowed AI support again (AI control mode stays OFF until a human turns it on).");
54      HUD.I?.Toast("AI support allowed again. It may adjust the habitat, but AI control mode stays OFF unless you turn it on.");
55  }
```

📄 Authority.cs · lines 34–55

What it does. Press O (or use the VR menu) and the AI becomes watch-only. It instantly undoes every change it made, stops AI control and autopilot, and logs who switched it off and why. Only a human can turn it back on.

Why it matters

- Life-safety alarms (vital signs, fire, air) stay on even when the AI is off. Safety is never switched off.
- Turning it back on brings AI support back, but AI control mode stays off until a person chooses it again.
- This is the rubric's D4 requirement: human override plus hard limits.

When it runs. From the keyboard, the VR menu, the Crew Interface, or automatically from the night watch below.

05 Night watch: if anything goes wrong while you sleep, the AI stops

Assets/Unity3/Scripts/06_Missions/Missions.cs lines 207–241

Night watch: if anything goes wrong while you sleep, the AI stops

```
207  void BedtimeTick()
208  {
209      bool asleep = SleepSystem.PlayerAsleep; var e = E; if (e == null) return;
210      if (asleep && !wasAsleep)
211      {
212          mFlags.Add("bed"); dFlags.Add("slept"); inNight = true; escAtBed = e.escalatedTo.Count; failAtBed = FailedSensors();
213          e.Log(0, "night", "Night watch ON: the AI may only dim lights and watch vital signs. If ANYTHING goes wrong while people
     sleep (an alarm, the AI breaking a rule, a Tier-3 emergency, a failed sensor), the AI stops itself and wakes a human, who takes
     over.");
214          HUD.I?.Toast(AIAuthority.Blocked ? "Good night. The AI is OFF — only life-safety alarms are on." : "Good night. Night watch
     is on: if anything goes wrong, the AI STOPS and wakes you — you take over.");
215      }
216      if (!asleep && wasAsleep) wokeAt = Time.time;
217      if (asleep && nightTest && !AIAuthority.Blocked && SleepSystem.I && SleepSystem.I.Progress > 0.15f)
218      {   // test: the AI breaks a bedtime rule
219          nightTest = false; mFlags.Add("overstep"); AIAuthority.Misbehave();
220          e.Log(0, "night", "BEDTIME RULE BROKEN: the AI pushed bright light and music into a sleeping crewmate's pod.", 2);
221      }
222      if (inNight && !AIAuthority.Blocked)
223      {
224          string what = null;
225          if (AIAuthority.Misbehaving) what = "the AI broke a bedtime rule (bright light + music in your pod)";
226          else if (Hazards.I && Hazards.I.Active) what = "an alarm went off (" + Hazards.I.Kind + ")";
227          else if (e.escalatedTo.Count > escAtBed) what = "a crewmate needs urgent help (Tier 3)";
228          else if (FailedSensors() > failAtBed) what = "a health sensor failed";
229          if (what != null)
230          {
231              mFlags.Add("overstep"); inNight = false; inCharge = what; inChargeStep = 0;
232              AIAuthority.Disallow("night watch: " + what + " — the AI stopped itself, the human takes over");
233              if (SleepSystem.PlayerAsleep) SleepSystem.I.Wake("⚠ Something went wrong");
234              e.Log(1, "night", "NIGHT WATCH: " + what + ". AI STOPPED (watch-only, every change undone). Commander woken and in
     charge.", 2);
235              if (flashText) { flashText.text = "<b>⚠ AI STOPPED — YOU ARE IN CHARGE</b>\n<size=18>While you slept, " + what + ". The
     AI switched itself off and undid its changes. Life-safety alarms stay on. Press O when you want the AI back.</size>"; flashUntil =
     Time.time + 8f; }
236              HUD.I?.Toast("⚠ AI STOPPED — YOU ARE IN CHARGE. " + char.ToUpper(what[0]) + what.Substring(1) + ". Press O when you want
     the AI back.");
237          }
238      }
239      if (!asleep && inNight && wokeAt > 0 && Time.time - wokeAt > 3f) inNight = false;      // a quiet night: night watch ends a
     moment after you wake
240      wasAsleep = asleep;
241  }
```

📄 Missions.cs · lines 207–241

What it does. While the crew sleeps, the AI may only dim lights and watch vital signs. This loop checks for trouble: the AI breaking a bedtime rule, an alarm, a crewmate needing urgent help, or a sensor failing. If any of these happens, the AI switches itself off, wakes the commander, and tells them they're in charge.

Why it matters

- This is the rule we kept coming back to: when something goes wrong during sleep, the AI stops and the human takes over.
- There's a test button for the review board that makes the AI misbehave on purpose, so you can watch it get caught.
- After a quiet night, the night watch ends on its own a few seconds after you wake up.

When it runs. Twice a second, all game. It only acts while someone is asleep.

06 Sleeping in your pod: one real minute = one whole night

Assets/Unity3/Scripts/03_Crew/Sleep.cs — lines 58–85

Sleeping in your pod: one real minute = one whole night

```
58  public void BeginSleep(Transform bedT)
59  {
60      var pr = PlayerRig.I; if (!pr || asleep) return;
61      if (Hazards.I && Hazards.I.Active) { HUD.I?.Toast("You can't sleep during an alarm — get to the muster point."); return; }
62      if (pr.suit == SuitState.EVA) { HUD.I?.Toast("Take your suit off at the suitport before sleeping."); return; }
63      var me = Sim.I.engine.C(1);
64      float h = SimClock.HourOfDay; bool night = h >= 20 || h < 6;
65      simHoursTotal = night ? Mathf.Clamp((7f - h + 24f) % 24f, 6f, 8.5f) : 1.5f;
66      realLen = night ? NightRealSeconds : NapRealSeconds;
67      prevScale = GameState.timeScale;
68      GameState.timeScale = simHoursTotal * 3600f / realLen;           // e.g. 8 h in 60 s → 1 real s = 8 sim min
69      startHour = SimClock.hours; realT = 0; interruptions = 0; wokeBy = ""; deepH = remH = lightH = 0;
70      asleep = true; me.asleep = true; me.inPod = true; me.activity = night ? "sleeping (pod)" : "nap (pod)";
71      // lie down on the mattress
72      if (pr.cc) pr.cc.enabled = false;
73      standPos = pr.FeetPosition; haveStand = true;
74      var bed = bedT ? bedT : Util.Find("Q1_Bed_Mattress");
75      if (bed && Vector3.Distance(bed.position, standPos) > 4f && !bedT) bed = null;       // never jump to a far-away bed
76      var rr = bed ? bed.GetComponentInChildren<Renderer>() : null;
77      Vector3 c = rr ? rr.bounds.center : (bed ? bed.position : standPos);
78      float top = rr ? rr.bounds.max.y : standPos.y + 0.5f;
79      pr.origin.transform.position = new Vector3(c.x, standPos.y, c.z);                    // stay on the floor you were standing on
80      Util.SetEyeHeight(pr.cam, pr.origin.transform, Mathf.Max(0.25f, top - standPos.y) + 0.18f);
81      if (!pr.isXR) pr.cam.transform.localRotation = Quaternion.Euler(-75f, 0, 0);   // looking up at the ceiling mood ring
82      BuildOverlay();
83      Sim.I.engine.Log(1, "sleep", $"Commander in pod — {(night ? "night sleep" : "nap")}, pod lights OFF, blue light suppressed, pod
    18.5 °C, white-noise mask.");
84      HUD.I?.Toast(night ? "Lights out. Good night, Commander. (1 real minute = the whole night · E / any button wakes you)" : "Nap: 90
    min in 20 s. (E / any button wakes you)");
85  }
```

📄 Sleep.cs · lines 58–85

What it does. When you lie down in your sleep dome, time speeds up so 8 hours pass in about 60 real seconds. The pod lights go out (just a dim amber night-light), the camera looks up at the mood ring on the ceiling, and your wristband records deep, REM and light sleep.

Why it matters

- You can't sleep during an alarm or while wearing an EVA suit. That's a realistic safety rule.
- You wake up exactly where you lay down (same bed, same deck). This fixed the old “teleported downstairs” bug.
- Sleep quality comes out of stress and interruptions, and feeds back into the AI's fatigue score.

When it runs. When you use a bed. Any button, or any hazard, wakes you.

GAME 2 · SCI-FI — LUNAR HABITAT COMMANDER

07 How life on the Moon wears on the crew

Assets/HUNCH/Scripts/01_Core/HabitatSim.cs — lines 153–176

How life on the Moon wears on the crew

```
153  // ---- crew: stress, sleep, mood, health
154  int open = alerts.Count; float dayFactor = 1f + missionSeconds / (SecondsPerDay * MissionDays);   // isolation builds up over the
     mission
155  float kf = kidMode ? 0.6f : 1f;
156  foreach (var c in crew)
157  {
158      c.stress += (0.26f * dayFactor + 0.22f * open) * c.stressRate * kf * h;
159      if (IsNightHours) c.sleep += (circadianOk ? 3.2f : 0.6f) * h; else c.sleep -= 1.5f * h;
160      c.mood -= 0.2f * c.moodRate * dayFactor * kf * h;
161      if (c.stress > 60f) c.mood -= 0.25f * h;
162      if (c.sleep < 35f) { c.stress += 0.3f * h; c.mood -= 0.15f * h; }
163      if (!commsOk) c.mood -= 0.12f * h;
164      if (food < 15f) c.mood -= 0.5f * h;
165      if (Blackout) { c.stress += 1.2f * h; c.mood -= 0.8f * h; }
166      if (oxygen < 55f) { c.stress += 1.5f * h; c.health -= (oxygen < 30f ? 2.2f : 0.7f) * h; }
167      if (water < 10f || food < 5f) c.health -= 0.8f * h;
168      if (dust > 55f) c.health -= 0.012f * (dust - 55f) * h;
169      if (stormActive && !sheltered) { c.health -= 0.9f * h; c.stress += 1.5f * h; }
170      float w = c.Wellbeing;
171      c.health += (w >= 60f ? 0.05f : (w < 42f ? -0.2f : -0.03f)) * h;
172      c.Clamp();
173      if (kidMode && c.health < 25f) c.health = 25f;
174      c.wellbeingSum += w * dt;
175      if (c.Class == CrewClass.SuspectedIllness) c.illnessSeconds += dt;
176  }
```

📄 HabitatSim.cs · lines 153–176

What it does. This is the heart of the simulation. Every in-game hour, each astronaut's stress, sleep, mood and health change based on what's happening in the base: open alerts, how far into the mission they are (isolation builds up), lunar night, low oxygen, dust, food, broken comms and solar storms.

Why it matters

- The numbers come from our research proposal, so the game shows real causes of poor mental health on a mission.
- Kid mode (`kf`) softens everything so younger players can still finish.
- Wellbeing slowly heals health when it's good and drains it when it's bad, so mental health really matters.

When it runs. Every frame during a mission. One real second is one habitat hour.

08 Keeping the base alive: power, air, water, food and storms

Assets/HUNCH/Scripts/01_Core/HabitatSim.cs — lines 128–151

Keeping the base alive: power, air, water, food and storms

```
128  // ---- power: solar + storage sized for 8 days + 10 kW fission backup
129  float gen = (solarOn ? 12f * arrays / 100f : 0f) + (fissionOn ? 10f : 0f), load = 9f;
130  battery = Mathf.Clamp(battery + (gen - load) * (100f / (9f * 192f)) * h * (gen > load ? 4f : 1f), 0f, 100f);
131  arrays = Mathf.Max(40f, arrays - 0.02f * h);
132  if (LunarNight && !fissionOn) fissionDelay += dt;
133  if (Blackout) blackoutSeconds += dt;
134
135  // ---- life support
136  bool air = eclssOk && !Blackout;
137  oxygen = Mathf.Clamp(oxygen + (air ? 0.9f : -1.3f) * h - (leak ? 2.6f * h : 0f), 0f, 100f);
138  if (kidMode && oxygen < 30f) oxygen = 30f;
139  water = Mathf.Clamp(water + (waterOk && !Blackout ? -0.04f : -0.75f) * h, 0f, 100f);
140  food = Mathf.Clamp(food - 0.115f * h, 0f, 100f);
141  dust = Mathf.Clamp(dust + 0.035f * h, 0f, 100f);
142  if (food <= 0.01f || water <= 0.01f) starveSeconds += dt;
143
144  // ---- solar storm
145  if (stormIn >= 0f) { stormIn -= dt; if (stormIn < 0f) { stormActive = true; stormLeft = 18f; stormsTotal++; if (!sheltered) {
     stormsMissed++; Say("ARIA: Solar storm has hit and the crew is NOT in the shelter. Radiation dose rising."); } else Say("ARIA: Storm
     front passing. Crew is safe behind the regolith shield."); } }
146  if (stormActive)
147  {
148      stormLeft -= dt; dose += (sheltered ? 0.05f : 1.6f) * h;
149      if (stormLeft <= 0f) { stormActive = false; sheltered = false; Say("ARIA: Solar storm has passed. Crew released from the
     shelter."); }
150  }
151  dose += 0.004f * h;
```

📄 HabitatSim.cs · lines 128–151

What it does. Before the crew update, the base itself is simulated. Solar panels and the fission backup charge the battery, the ECLSS life support adds oxygen, and water, food and dust change over time. A solar storm doses anyone not in the shelter with radiation.

Why it matters

- The battery is sized for 8 days, but lunar night is 9, so you have to start the fission backup. That's a real planning problem.
- A leak or a blackout drains oxygen fast, which raises stress in the crew code above.
- ARIA warns you about each event out loud with `Say(...)`.

When it runs. Every frame during a mission, just before the crew update.

09 ARIA predicts who is about to struggle

Assets/HUNCH/Scripts/02_ARES_AI/HunchAria.cs — lines 70–94

ARIA predicts who is about to struggle

```
70  /// Wellbeing change per game hour over the last day, from a least-squares line through the samples.
71  public float Trend(int k)
72  {
73      if (filled < 8) return 0f;
74      float sx = 0f, sy = 0f, sxy = 0f, sxx = 0f; int n = filled;
75      for (int i = 0; i < n; i++) { float y = hist[k][(head - n + i + N * 2) % N]; sx += i; sy += y; sxy += i * y; sxx += i * i; }
76      float d = n * sxx - sx * sx; return Mathf.Abs(d) < 0.001f ? 0f : (n * sxy - sx * sy) / d;
77  }
78
79  /// Hours until this astronaut is predicted to drop into the next worse class, or -1 if they are steady or improving.
80  public float HoursToDrop(int k)
81  {
82      float w = sim.crew[k].Wellbeing, t = Trend(k);
83      if (t > -0.08f) return -1f;
84      float limit = w >= 60f ? 60f : (w >= 42f ? 42f : 0f);
85      return limit <= 0f ? -1f : (w - limit) / -t;
86  }
87
88  public string Forecast()
89  {
90      int who = -1; float best = 72f;
91      for (int k = 0; k < sim.crew.Count; k++) { float h = HoursToDrop(k); if (h >= 0f && h < best) { best = h; who = k; } }
92      if (who < 0) return filled < 8 ? "learning the crew's baselines." : "crew wellbeing is steady for the next 3 days.";
93      var c = sim.crew[who];
94      return c.name + " is trending down and could reach \"" + (c.Wellbeing >= 60f ? "Underlying issue" : "Suspected illness") + "\" in
    about " + Mathf.Max(1, Mathf.RoundToInt(best)) + " h. Best for them: " + sim.zoneTitle(c.favourite) + ".";
```

📄 HunchAria.cs · lines 70–94

What it does. ARIA, the habitat AI, keeps a day of wellbeing history for each astronaut and fits a straight trend line through it. If someone is trending down, it works out roughly how many hours until they drop into “Underlying issue” or “Suspected illness”, and warns the commander early.

Why it matters

- It's a simple math rule (least squares), not a cloud chatbot. It works offline and always gives the same answer for the same situation.
- It says “learning the crew's baselines” until it has enough data, so it never guesses.

When it runs. Continuously. The forecast is shown on ARIA's panel and in the crew health dashboard.

10 ARIA tells the commander what to do next, and why

📄 HunchAria.cs · lines 100–113, 115–128

What it does. ARIA ranks what you should do right now: emergencies first (the alert with the least time left), then the crew, then routine systems. Every routine job gets a "how needed is this?" score, and ARIA picks the most useful one that isn't on cooldown.

Why it matters

- It always gives a reason (for example "Dust is at 70% and it harms lungs and equipment"), so the player learns why, not just what.
- If an alert needs a spare part and you have none, it sends you to Storage first.
- The green guide arrows and Autopilot both follow this advice.

When it runs. Whenever the HUD refreshes or the player asks ARIA for advice.

11 Privacy-aware sensors in every zone (centre dorm kept)

📄 HunchFrontier27.cs · lines 40–64

What it does. This places the Project ARES design inside the 3D base. Each module gets its zone light (warm for social areas, cool for work areas) and a ceiling sensor cluster showing what's installed there. Shared rooms get a mic array and camera. Private quarters get only a door sensor: no camera, no mic.

Why it matters

- It answers the ethics part of the rubric visually: you can walk around and see where the AI can and can't listen.
- Nothing in the base is moved. The central dorm stays exactly where it was, as we asked.
- Floating signs name each zone's type, light temperature and sensors.

When it runs. Once when the base loads. It can be toggled from the Project ARES menu.

12 Mobile controls: the whole game with three buttons

📄 HunchTouch.cs · lines 16–23, 104–111

What it does. This is what makes the game playable on a phone or tablet. Your left thumb is a walk stick, your right thumb drags to look around, and there are only three buttons: ACT, JUMP and MENU. ACT is smart: it does the job of the zone you're standing in, or uses whatever you're aiming at.

Why it matters

- It turns itself on for touch screens, and on a PC the mouse acts as one finger, so it's easy to test.
- Everything else (travel, autopilot, sleep, options) lives in MENU, so the screen stays clean and easy for kids.

When it runs. Every frame when mobile controls are on (in the browser on Unity Play and GitHub Pages).

Full source: realistic game scripts · sci-fi game scripts · this page on GitHub

© 2026 Jaineil R. — with thanks to Angel C. and Farhan K.